Furuno DRS4D-NXT Radar

Wire Protocol & System Technical Manual

J.C. Vaught

March 2026

A complete technical reference for the Furuno DRS4D-NXT marine radar system, covering the wire protocol, echo data encoding, system architecture, web interface, and development guide.

Contents

I Overview & Usage **6**

1 Introduction **6**
1.1 Hardware Overview . . . 6
1.2 Software Components . . . 6

2 Quick Start & Installation **8**
2.1 Prerequisites . . . 8
2.2 Building . . . 8
2.3 First Run . . . 8
2.4 Auto-Discovery . . . 8

3 CLI Modes **10**
3.1 General Syntax . . . 10
3.2 Web Server Mode . . . 10
3.3 Discover Mode . . . 10

4 Web Interface **11**
4.1 Starting the Server . . . 11
4.2 Page Tour . . . 11
4.2.1 Controls (index.html) . . . 11
4.2.2 Radar View (radar.html) . . . 12
4.2.3 Settings (settings.html) . . . 13
4.2.4 System Status (status.html) . . . 15
4.3 PPI Display . . . 16

5 Recording & Data Formats **17**
5.1 Starting and Stopping Recording . . . 17
5.2 Output Directory Structure . . . 17
5.3 CSV Format . . . 17
5.4 Settings Snapshot . . . 18

6 Pattern System **19**
6.1 What Patterns Do . . . 19
6.2 Simple Pattern Format (7-Field) . . . 19
6.3 Advanced Pattern Format (13-Field) . . . 19
6.4 Pattern Queue . . . 20
6.5 Loading Patterns . . . 20
6.6 NaN Override Behavior . . . 20

II Software Internals **21**

7 System Architecture Overview **21**
7.1 Module Summary . . . 21

7.2 System-Level Block Diagram . . . 21
7.3 Module-Level Diagram . . . 22
7.4 Data Flow Diagram . . . 22
7.5 Concurrency Model . . . 22

8 PPI Rendering **24**
8.1 Coordinate Transformation . . . 24
8.2 Color Mapping . . . 24
8.3 Overlay Elements . . . 25
8.4 PNG Encoding . . . 25
8.5 How to Modify . . . 25

9 Web Server **26**
9.1 Framework . . . 26
9.2 Shared State . . . 26
9.3 Route Registration . . . 27
9.4 Handler Pattern . . . 27
9.5 Echo Listener Thread . . . 27
9.6 Static File Serving . . . 28
9.7 How to Add a New Endpoint . . . 28

10 REST API Reference **29**
10.1 Power and Connection . . . 29
10.2 Settings . . . 29
10.3 State Queries . . . 29
10.4 Echo Image . . . 30
10.5 Recording . . . 30
10.6 Pattern Execution . . . 30
10.7 Stub Endpoints . . . 31

11 Recording System Internals **32**
11.1 Echo Listener Integration . . . 32
11.2 Directory Creation . . . 32
11.3 CSV Writing . . . 32
11.4 Settings Snapshots . . . 32
11.5 Setting Change History . . . 33

12 Pattern Execution Engine **34**
12.1 Step Parsing . . . 34
12.2 Differential Command Generation . . . 34
12.3 Single Pattern Execution . . . 34
12.4 Queue Execution . . . 34
12.5 NaN Override Logic . . . 35
12.6 Stop Mechanism . . . 35

13 CLI Modes (All Other Modes) **36**
13.1 Handshake . . . 36
13.2 TX On / TX Off . . . 36
13.3 Full Cycle . . . 36

13.4 Render Live . . . 36
13.5 Echo Listen . . . 36
13.6 Command Test . . . 37
13.7 Dump Raw . . . 37
13.8 Test Commands . . . 37
13.9 Validate . . . 37

14 Connection State Machine **38**
14.1 States and Transitions . . . 38
14.2 RadarError Enum . . . 38
14.3 Timeout Values . . . 39
14.4 Graceful Degradation . . . 39
14.5 Keepalive Handling . . . 39

15 Testing & Validation **40**
15.1 Running Tests . . . 40
15.2 Test Categories . . . 40
15.2.1 Connection Tests (6) . . . 40
15.2.2 TX Tests (3) . . . 40
15.2.3 Settings Tests (14) . . . 40
15.2.4 Echo Impact Tests (2) . . . 40
15.2.5 Render Integration Test (1) . . . 41
15.3 A/B Command Testing . . . 41
15.4 Validation Suite . . . 41

III Echo Data Pipeline **42**

16 Echo Data Format **42**
16.1 Packet Structure . . . 42
16.2 Header Field Breakdown . . . 42
16.3 Echo Metadata Extraction . . . 42
16.4 Spoke Sub-Header . . . 42
16.5 Multi-Spoke Packing . . . 42
16.6 Azimuth and Heading . . . 42
16.7 Typical Packet Characteristics . . . 42

17 Echo Encoding Types **43**
17.1 Encoding 0: Raw Uncompressed . . . 43
17.2 Encoding 1: RLE with Repeat Marker . . . 43
17.3 Encoding 2: Delta RLE with Previous Spoke Reference . . . 43
17.4 Encoding 3: 2-Bit RLE with Dual Reference . . . 43
17.5 Encoding Selection . . . 43
17.6 Display Value Conversion . . . 43
17.7 Rotation Detection . . . 43

18 Doppler Processing **44**
18.1 Status Field Encoding . . . 44
18.2 Echo Byte Encoding . . . 44

18.3 Downstream Usage . . . 44
18.4 Practical Notes . . . 44

19 Dual-Pulse Interleaving **45**
19.1 Pulse Types . . . 45
19.2 Range Dependence . . . 45
19.3 Rez Boost Effect . . . 45
19.4 Spoke Handling . . . 45

IV Network & Wire Protocol **46**

20 Network Architecture **46**
20.1 Topology . . . 46
20.2 Port Map . . . 46
20.3 Auto-Discovery and Network Configuration . . . 46
20.3.1 Platform-Specific IP Alias . . . 46

21 Control Protocol **47**
21.1 Message Format . . . 47
21.2 Binary Keepalive . . . 47
21.3 Binary vs ASCII Disambiguation . . . 47
21.4 Response Types . . . 47
21.5 Response Parsing Pipeline . . . 47
21.6 Command Acknowledgment . . . 47
21.7 Control Channel Drain . . . 47
21.8 Initialization Sequence . . . 47
21.8.1 Phase 1: Discovery . . . 47
21.8.2 Phase 2: Authentication . . . 47
21.8.3 Phase 3: Control Channel . . . 47
21.8.4 Phase 4: Firmware Query . . . 47
21.8.5 Phase 5: Register Dump . . . 47
21.8.6 Phase 6: Steady State . . . 47

22 Command Reference **48**
22.1 TX State (`$S69`) . . . 48
22.2 Range (`$S62`) . . . 48
22.3 Gain (`$S63`) . . . 48
22.4 Sea Clutter (`$S64`) . . . 48
22.5 Rain Clutter (`$S65`) . . . 48
22.6 Pulse Width (`$S68`) . . . 48
22.7 STC Near (`$S85`) . . . 48
22.8 STC Middle (`$S86`) . . . 48
22.9 STC Far (`$S87`) . . . 48
22.10 Main Bang Suppression (`$S83`) . . . 48
22.11 Antenna Height (`$S84`) . . . 48
22.12 Antenna Revolution (`$S89`) . . . 48
22.13 Interference Rejection (`$S67`, param[2]=2) . . . 48

22.14Noise Reduction (`$S67`, param[2]=3) . . . 48
22.15Rez Boost (`$SEE`) . . . 48
22.16Bird Mode (`$SED`) . . . 48
22.17Doppler / Target Analyzer (`$SEF`) . . . 48
22.18No-Transmit Sectors (`$S77`) . . . 48
22.19Query Commands . . . 48

23 Periodic Telemetry **49**
23.1 Telemetry Message Table . . . 49
23.2 Telemetry Processing . . . 49
23.3 Parsed vs Discarded Data . . . 49

24 Proprietary Library Reference **50**
24.1 Module Breakdown . . . 50
24.2 Shared Command IDs . . . 50

Part I

Overview & Usage

1 Introduction

This document is the complete technical reference for the Furuno DRS4D-NXT marine radar system as interfaced through the `furuno-server` Rust application. It covers radar operation via a browser-based web interface, the internal software architecture, echo data encoding and decoding, and the full wire protocol. The intended audiences are new users learning to operate the system, contributors modifying the codebase, and developers building their own applications on top of the radar hardware.



Figure 1: System overview. The radar scanner communicates with the Rust server over a dedicated Ethernet link. The server exposes a web interface accessible from any browser on the network.

1.1 Hardware Overview

The Furuno DRS4D-NXT is a solid-state Doppler marine radar operating in the X-band at 9.41 GHz. The scanner unit contains the antenna, transmitter, and receiver in a single sealed radome enclosure. It connects to a host processor via a standard Ethernet cable. The scanner is assigned a fixed IP address of `172.31.3.226` on a `172.31.0.0/16` subnet.

The radar transmits at up to 25 watts using solid-state amplification rather than a traditional magnetron, which eliminates warm-up time and provides consistent performance across the operating temperature range. The antenna rotates at either 24 or 48 RPM depending on the selected speed mode. Echo data is delivered as radial spokes, with 8192 unique azimuth positions per full rotation and up to 868 range bins per spoke. The system supports six pulse width settings (S1, S2, M1, M2, M3, L), 18 range scales from 1/16 NM to 48 NM, and built-in Doppler target discrimination.

1.2 Software Components

The system consists of three software components.

furuno-server is a Rust binary that implements the complete Furuno wire protocol from scratch. It handles network discovery, authentication, control commands, echo data reception and decoding across four encoding types, PPI (Plan Position Indicator) rendering, and serves a web interface for browser-based operation. The application is built with Axum 0.7 for the HTTP server and Tokio for the async runtime. All radar communication—TCP control, UDP multicast echo reception, and

subnet management—is handled natively without any Furuno proprietary libraries.

The *web interface* is a set of static HTML, CSS, and JavaScript files served by the Rust application. It provides four pages: a controls dashboard for quick radar operation, a full-screen PPI display with history compositing, a detailed settings panel, and a system status view. The interface communicates with the server through a REST API and polls for updated radar images at 500 ms intervals.

The *documentation* consists of this technical manual and inline source code comments. The manual is maintained as a LaTeX file with diagrams compiled as standalone PDFs from TikZ source files in the `figures/` directory.

2 Quick Start & Installation

2.1 Prerequisites

The Rust toolchain is required to build the application. Install it via `rustup` if it is not already present.

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
source ~/.cargo/env
```

The application builds and runs on Linux (x86_64 and aarch64), macOS, and Windows. No external C libraries or Furuno SDKs are required.

2.2 Building

From the repository root, build the release binary.

```
cd furuno-server
cargo build --release
```

The compiled binary is located at `furuno-server/target/release/furuno-server`. A release build is recommended for deployment as it enables compiler optimizations that significantly improve echo decoding and PPI rendering throughput.

2.3 First Run

Start the web server with the `web` mode.

```
cargo run --release -- web
```

The web interface is then available at `http://localhost:8080` in any browser. On startup, the application attempts to discover the radar automatically (see below). If the radar is not reachable, the web server starts anyway and serves a degraded interface. The `/reconnect` endpoint can be used to retry the connection later.

2.4 Auto-Discovery

The application resolves the radar and host IP addresses through a three-stage fallback process.

First, it checks for explicit command-line flags. If `--radar-ip` is provided, that address is used directly. The `--host-ip` flag is optional; if omitted, the application determines a suitable host address on the radar's subnet automatically.

```
cargo run --release -- --radar-ip 172.31.3.226 web
```

Second, if no flags are provided, the application broadcasts UDP discovery beacons on port 10010 across all detected network interfaces and listens for 3 seconds. If the radar responds, both the radar and host addresses are extracted from the response.

Third, if discovery fails, the application falls back to the default addresses: `172.31.3.226` for the radar and `172.31.1.100` for the host. If the host does not already have an interface on the `172.31.0.0/16` subnet, a secondary IP alias is added to the physical Ethernet interface automatically. On Linux this uses `ip addr add`; on macOS, `ifconfig alias`.

3 CLI Modes

3.1 General Syntax

The application accepts optional IP flags followed by a mode name and mode-specific arguments.

```
furuno-server [--radar-ip IP] [--host-ip IP] <mode> [args...]
```

The two user-facing modes are `web` and `discover`. Additional developer-oriented modes (`handshake`, `tx-on`, `tx-off`, `echo`, `full`, `render-live`, `dump-raw`, `test-commands`, `validate`) are documented in §13. If no mode is given, the application defaults to `handshake`.

3.2 Web Server Mode

The `web` mode starts the HTTP server and is the primary way to operate the radar.

```
furuno-server web [port] [webdir]
```

The optional `port` argument sets the listening port (default 8080). The optional `webdir` argument specifies the path to the static web files (default `../web` relative to the binary). On startup, the server attempts to authenticate with the radar and initialize the control channel. If the radar is unreachable, the server starts anyway with degraded functionality—the PPI display shows a blank image and setting changes are queued until a connection is established via the `/reconnect` endpoint.

3.3 Discover Mode

The `discover` mode broadcasts UDP beacons on port 10010 across all network interfaces and prints any radar responses it receives.

```
furuno-server discover
```

This is useful for determining the radar's IP address on an unfamiliar network or verifying that the radar is powered on and reachable. The mode runs until a response is received or the timeout expires.

4 Web Interface

4.1 Starting the Server

The web server starts when the application is launched in `web` mode (see §3.2). By default it listens on port 8080 and serves static files from the `../web` directory relative to the binary. Both values can be overridden with positional arguments. The server starts even if the radar is not reachable, allowing the interface to load and wait for a connection. The `/reconnect` API endpoint triggers a fresh authentication and initialization attempt without restarting the server.

4.2 Page Tour

The web interface consists of four HTML pages, all sharing a common header with navigation links, a consistent CSS stylesheet, and a single shared JavaScript file (`script.js`).

4.2.1 Controls (index.html)

The controls page is the default landing page and provides the most commonly used radar operations. It is divided into two panels. The left panel displays a radar preview image (refreshed every 2 seconds) with a link to the full-screen radar view. The right panel contains quick controls for TX power (on/off), CSV recording (start/stop), range selection (17 scales from 0.0625 to 36 NM), pulse width (S1 through L), manual gain (0–100), and RezBoost level (off/low/medium/high). Each control has an apply button and a refresh button that queries the radar's current state.

Below the quick controls is a pattern loader panel. Users can load a pattern file (`.txt` or `.csv`), set a repeat count (-1 for infinite looping), assign an optional label, and add the pattern to a queue. The queue is displayed as an ordered list, and patterns execute sequentially when the RUN button is pressed. A STOP button halts execution mid-pattern, and RESET clears the queue.

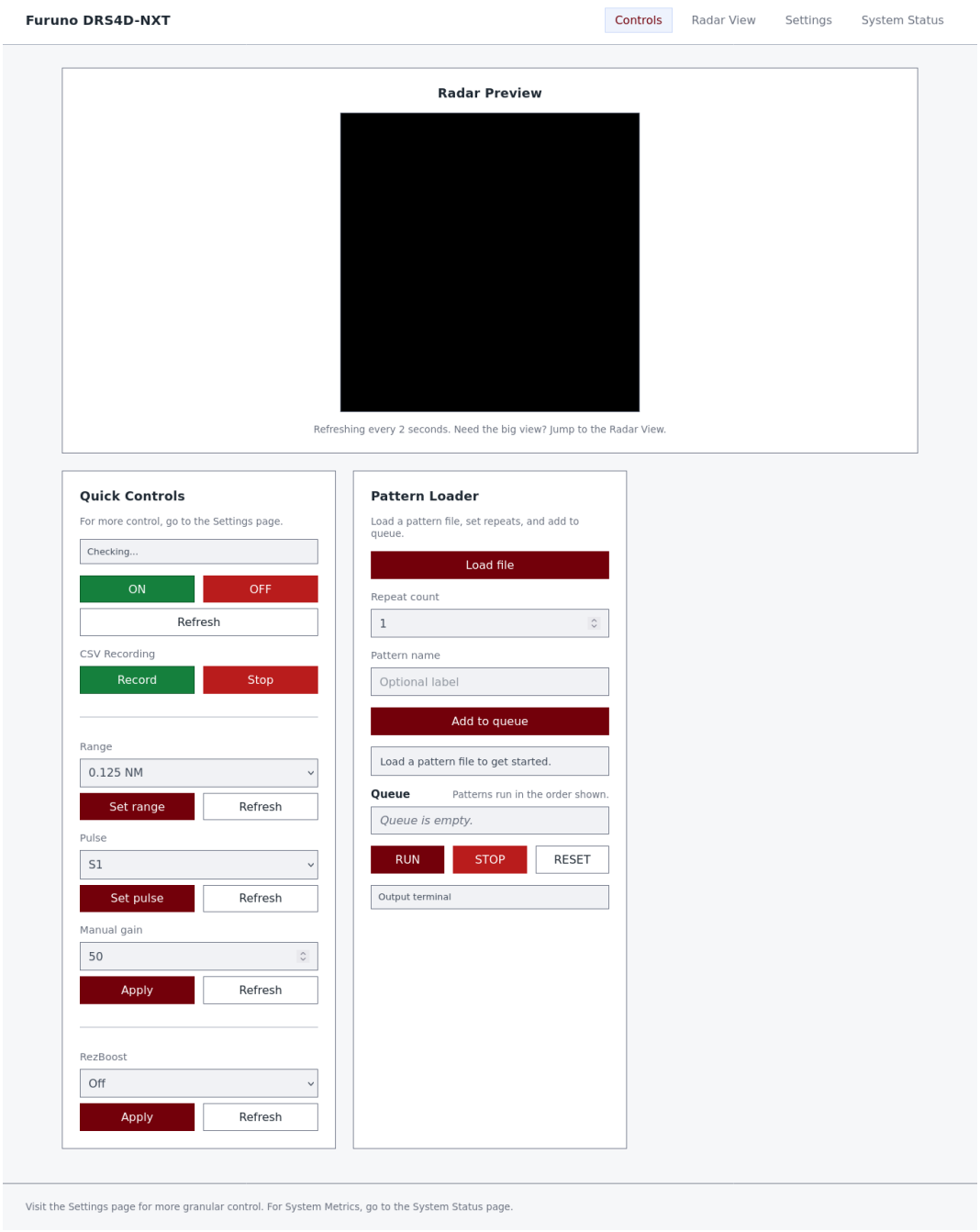


Figure 2: Controls page (`index.html`). The radar preview is shown at top, with quick controls and the pattern loader below.

4.2.2 Radar View (radar.html)

The radar view page provides a full-screen PPI display. The primary display element is an HTML5 canvas (1024×1024 pixels) that composites multiple frames of radar data to show target history. Below the canvas are controls for track length (1–50 frames), current frame color, and track history color, with a reset button to restore defaults. An `<img>` element behind the canvas serves as a

fallback if canvas rendering is unavailable. The canvas is refreshed every 500 ms by fetching the latest PPI image from the `/newest_png` endpoint.

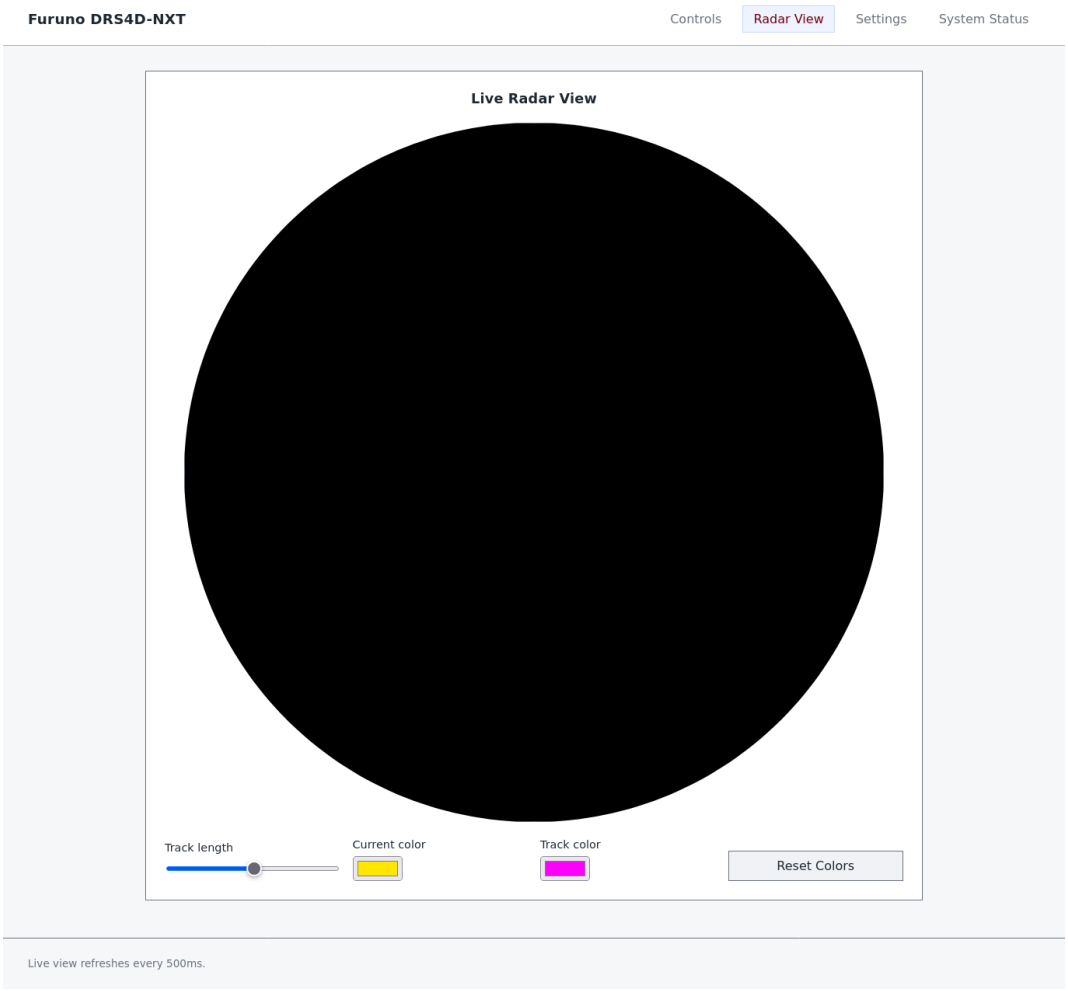


Figure 3: Radar view page (`radar.html`). The PPI display with history compositing and track color controls.

4.2.3 Settings (settings.html)

The settings page provides granular control over all radar parameters. It is organized into panels grouped by function.

The *connection panel* shows the current connection status and provides reconnect and refresh buttons. A small radar preview image is displayed alongside.

The *power control panel* provides TX on/off buttons and a refresh button to query the current transmitter state.

The *range and pulse panel* allows independent or combined adjustment of range and pulse width, with the same options available on the controls page.

The *gain panel* supports both manual and automatic gain modes. In manual mode, the user sets

a value from 0 to 100. In auto mode, a bias offset (0–100) adjusts how aggressively the automatic gain control responds.

The *STC panel* exposes three sensitivity time control zones—near, mid, and far—each with an independent numeric input.

The *signal processing panel* groups main bang suppression (0–255), interference rejection (on/off), bird mode (off/low/medium/high), and Doppler target analysis (off, internal heading, external heading).

The *clutter panel* provides sea clutter (A/C SEA) and rain clutter (A/C RAIN) controls, each with manual and automatic modes. Manual mode sets a fixed rejection level; auto mode adjusts a bias for the radar's internal adaptive filter.

The *antenna speed panel* controls rotation speed (auto/low/high) and a high-RPM override toggle.

The *RezBoost and display panel* sets the resolution enhancement level and display parameters including sweep line count and interpolation mode.

The *pattern loader* on this page has the same functionality as the one on the controls page, with the addition of a manual text input option for entering pattern steps directly.

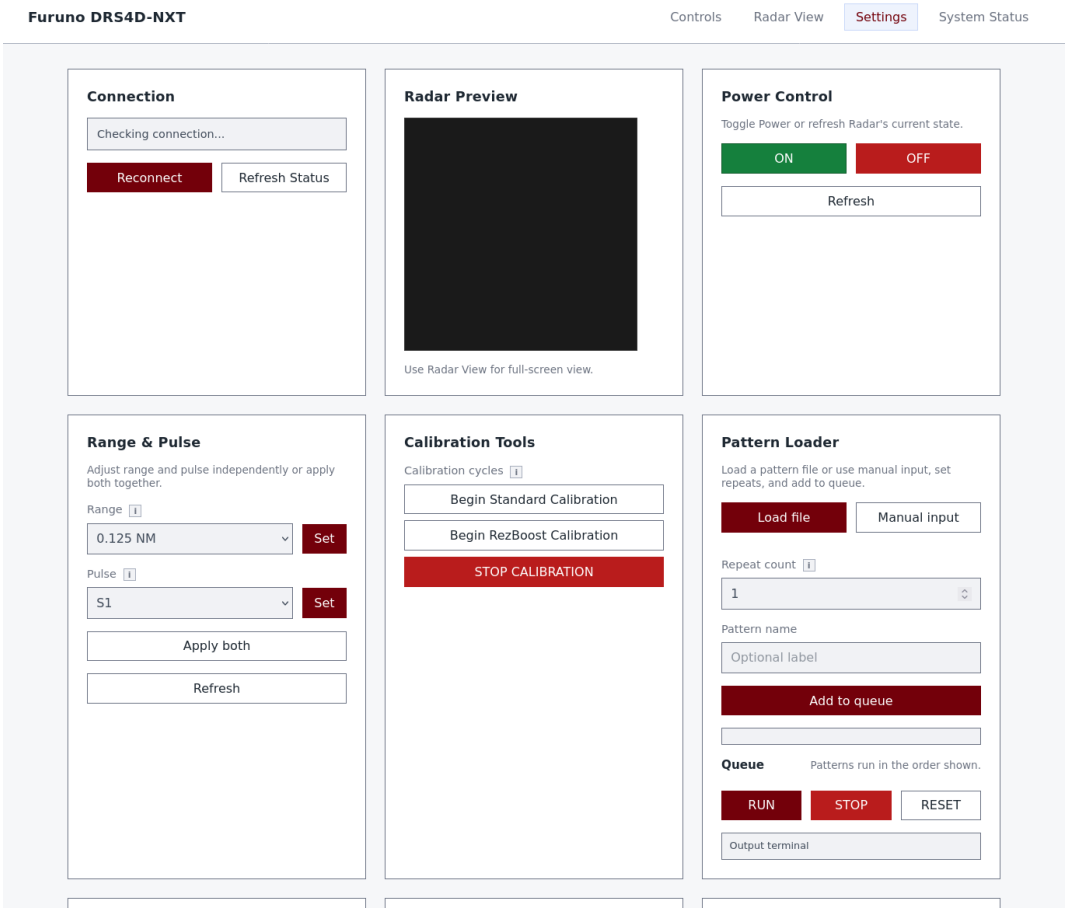


Figure 4: Settings page (`settings.html`), upper half. Connection, radar preview, power control, range/pulse, calibration tools, and pattern loader panels.

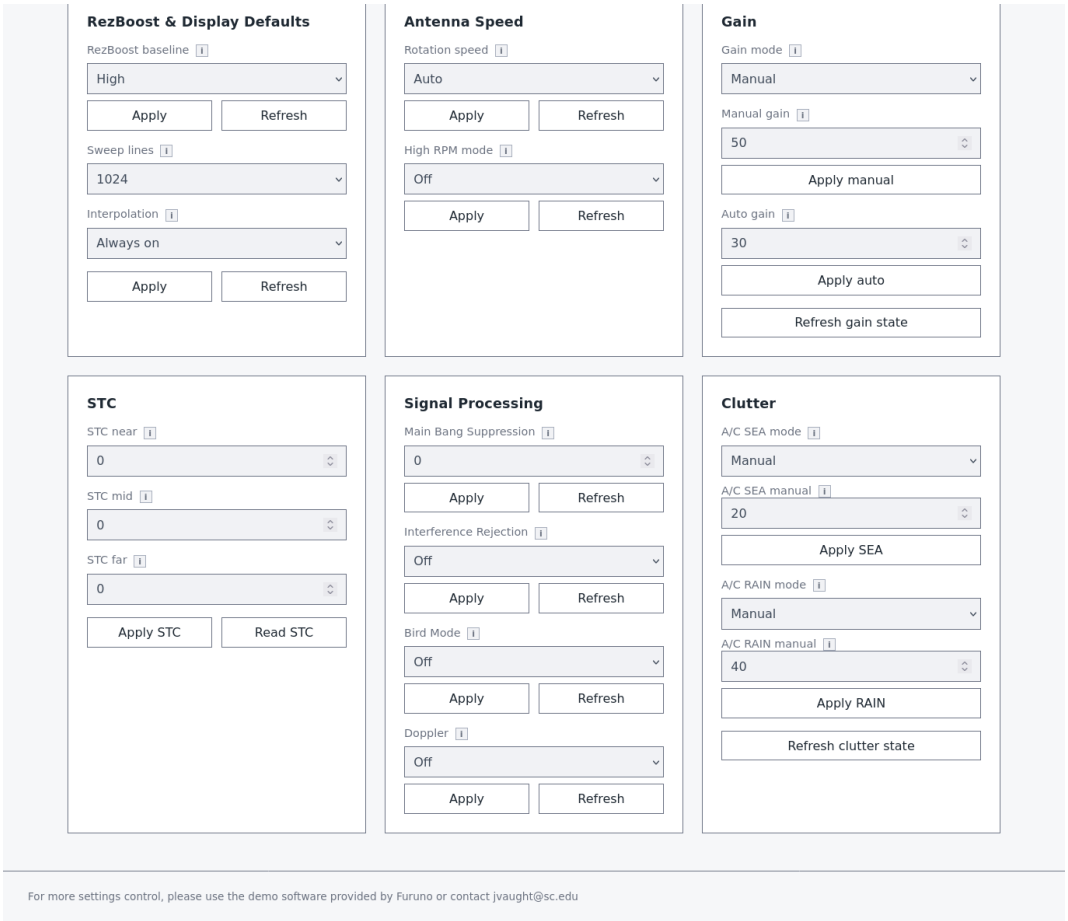


Figure 5: Settings page (`settings.html`), lower half. RezBoost, antenna speed, gain, STC, signal processing, and clutter panels.

4.2.4 System Status (status.html)

The system status page displays live telemetry from both the radar and the host system, updated every 500 ms.

The *radar connection* panel shows connection status, radar IP, host IP, TX state, and firmware version. The *session* panel shows server uptime, total rotations received, recording state, the current recording directory, and pattern queue status. The *active settings* panel displays a read-only summary of all current radar parameters: range, pulse, gain, RezBoost, antenna speed, Doppler, interference rejection, bird mode, main bang suppression, STC values, and clutter settings.

The *host resources* panels display CPU load (overall and per-core), memory usage, storage usage, network throughput, and CPU temperature. Each metric includes a progress bar and numeric label.

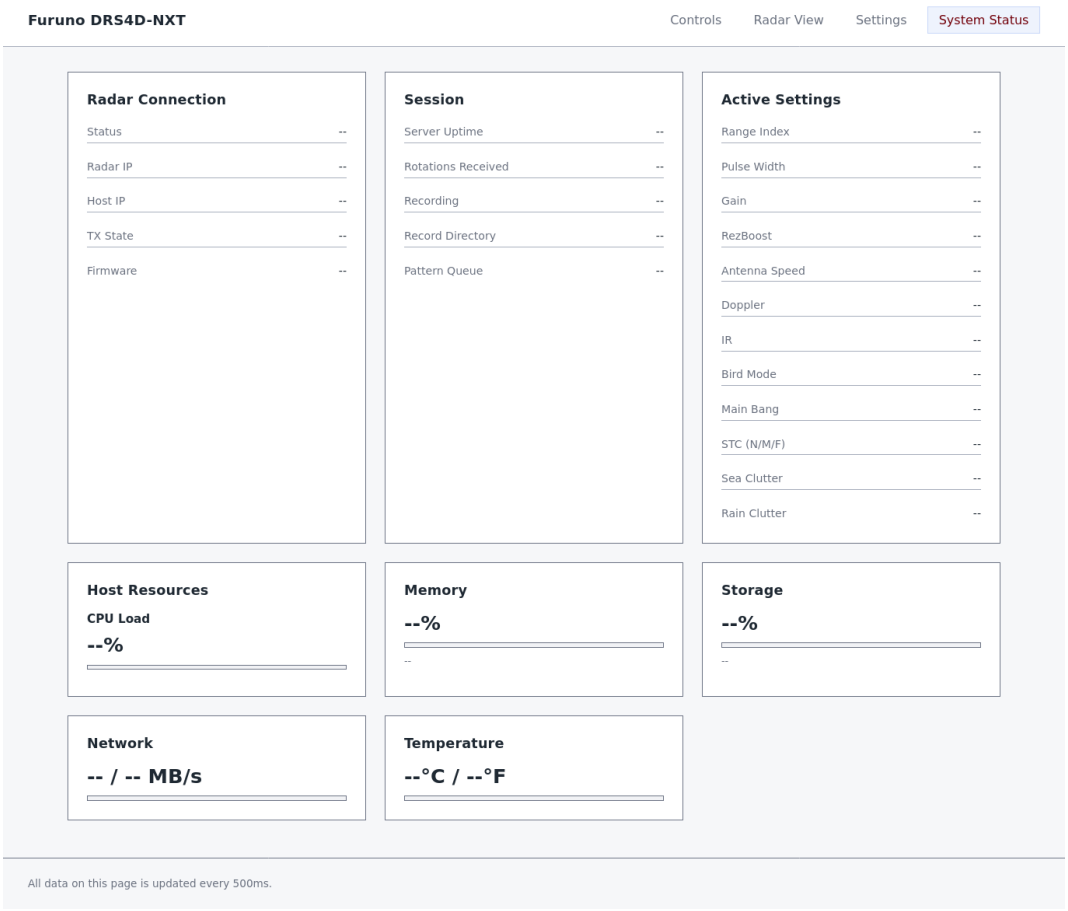


Figure 6: System status page (`status.html`). Live telemetry from the radar and host system.

4.3 PPI Display

The PPI (Plan Position Indicator) display renders radar echo data as a top-down circular image with the radar at the center. The server renders each complete antenna rotation into a 640×640 pixel PNG image using the warm-tinted color mapping described in §8.2. The browser fetches the latest image via the `/newest_png` endpoint.

On the radar view page, the JavaScript composites multiple frames onto an HTML5 canvas to show target motion history. The *track length* slider controls how many previous frames are retained (1–50). The *current color* picker tints the most recent frame, and the *track color* picker tints older frames. Older frames are drawn with decreasing opacity to produce a fading trail effect. This compositing runs entirely in the browser; the server delivers only single-frame PNG images.

5 Recording & Data Formats

5.1 Starting and Stopping Recording

Recording is toggled through the web interface or by calling the `/toggle_recording` API endpoint with a `state` parameter of `start` or `stop`. When recording starts, the server creates a new timestamped directory under `recordings/` with `csv/` and `png/` subdirectories. If a directory with the same timestamp already exists, a numeric suffix is appended (e.g., `_01`, `_02`). Each complete antenna rotation produces one CSV file and one PNG file, written automatically by the echo listener thread. When recording is stopped, a `settings.json` snapshot is written to the recording directory.

5.2 Output Directory Structure

Each recording session produces the following directory structure.

```
recordings/<timestamp>/
  csv/rotation_<ts>.csv
  png/rotation_<ts>.png
  settings.json
```

The top-level directory is named with a compact timestamp (e.g., `20260307_143022`). For pattern-based recordings, the sanitized pattern label is appended (e.g., `20260307_143022_sweep1`). The `csv/` and `png/` subdirectories are created on demand when the first rotation is written.

5.3 CSV Format

Each CSV file represents one complete antenna rotation. The file begins with a metadata header of `#`-prefixed comment lines that capture all radar settings at the time of recording.

```
# range_index,4
# gain_auto,false
# gain_value,50
# pulse_width,0
# sea_clutter,0
# rain_clutter,0
# rezboost,0
# interference_rejection,false
# bird_mode,0
# doppler,0
# antenna_speed,0
# main_bang,0
# near_stc,0
# middle_stc,0
# far_stc,0
Status  Scale   Range   Gain    Angle   EchoValues
0   4   4   19  0   12,14,8,...
0   4   4   19  1   10,11,9,...
```

After the header, the column line and data rows are tab-separated. Each row represents one spoke and contains a status field (always 0), the range scale index, the range index, the gain percentage (mapped from the 0–255 wire range to 0–100), the azimuth angle in ticks (0–8191), and the echo bin values as a comma-separated list of integers.

5.4 Settings Snapshot

When a recording session ends, the server writes a `settings.json` file to the recording directory. The snapshot format depends on how the recording was initiated.

For manual recordings, the snapshot contains the mode (`"manual"`), the recording directory path, the full `CurrentSettings` object, and a `history` array of `SettingChange` entries. Each history entry records a timestamp, the setting name, and the new value, capturing every parameter change made during the recording session.

For pattern-based recordings, the snapshot additionally includes the `PatternInfo` object describing the pattern that was executing, including all pattern steps with their parameter values and repeat counts.

6 Pattern System

6.1 What Patterns Do

Patterns automate sequences of radar parameter changes. A pattern consists of one or more *steps*, each specifying a set of radar settings and a *repeat count* that determines how many full antenna rotations that step is held before advancing to the next. When recording is active during pattern execution, each rotation produces a CSV and PNG file, allowing systematic data collection across parameter combinations.

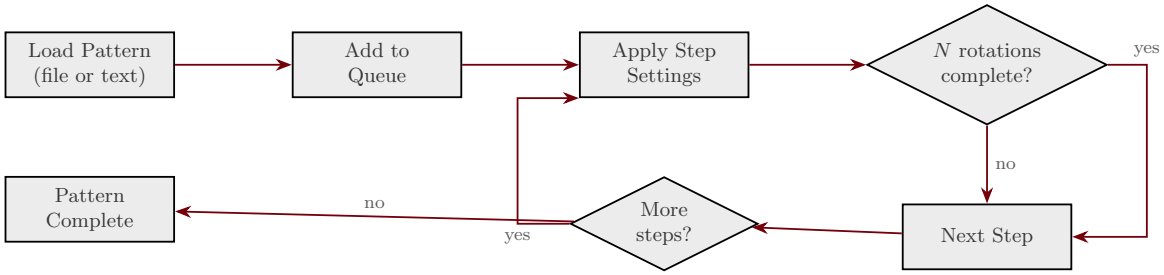


Figure 7: Pattern execution flow. Each step is applied to the radar and held for the specified number of antenna rotations before advancing.

6.2 Simple Pattern Format (7-Field)

The simple format uses seven comma-separated fields per step. All fields are required and all values are set explicitly.

```
range, gain, pulse, rezboost, speed, interp, repeat
```

For example, the step `4,50,0,3,2,1,5` sets range index 4 (1 NM), gain 50, pulse S1, RezBoost high, auto antenna speed, interpolation on, and holds for 5 rotations. Multiple steps are separated by semicolons.

```
4,50,0,3,2,1,5; 8,50,2,3,2,1,5; 11,50,4,3,2,1,5
```

6.3 Advanced Pattern Format (13-Field)

The advanced format extends the simple format with six additional fields for fine-grained signal processing control.

```
range, gain, pulse, rezboost, speed, interp,
    sea_clutter, rain_clutter, main_bang,
    interference_rejection, bird_mode, doppler, repeat
```

Any of the 12 parameter fields (all except `repeat`) can be set to `NaN` to leave that setting unchanged from its current value. This is particularly useful when only a subset of parameters needs to vary

across steps. For example, the step `4,NaN,0,NaN,NaN,NaN,NaN,NaN,NaN,NaN,NaN,NaN,3` changes only the range to index 4 and the pulse to S1, leaving all other settings at their current values and holding for 3 rotations.

6.4 Pattern Queue

Multiple patterns can be loaded into a queue and executed sequentially. Each queue entry has its own pattern data, repeat count, and optional label. The repeat count controls how many times the entire pattern is cycled (-1 for infinite looping). Patterns execute in the order they were added.

During queue execution, each pattern item creates its own recording session with a separate timestamped directory (tagged with the sanitized pattern label if one was provided). The queue can be stopped at any time using the STOP button, which halts execution between steps. A RESET button clears the entire queue.

6.5 Loading Patterns

Patterns can be loaded in two ways. The first is file upload: clicking the "Load file" button opens a file picker that accepts `.txt` or `.csv` files. The file contents are parsed as semicolon-separated steps. The second is manual input: on the settings page, clicking "Manual input" reveals a text area where steps can be typed or pasted directly.

After loading, the user sets a repeat count and an optional label, then clicks "Add to queue" to append the pattern to the queue list. Multiple patterns can be queued before pressing RUN.

6.6 NaN Override Behavior

When a pattern step uses `NaN` for a field, that setting remains *unlocked* during execution. This means the user can manually change that parameter through the web UI while the pattern runs, and the change takes effect immediately. Conversely, parameters with explicit values in the current step are *locked*—manual changes to locked parameters are rejected with an error message indicating that the setting is controlled by the active pattern.

This mechanism allows hybrid operation where some parameters are swept by the pattern while others are adjusted interactively. For example, a pattern that sweeps range and pulse but sets all clutter fields to `NaN` lets the operator tune clutter rejection in real time while the automated sweep runs.

Part II

Software Internals

7 System Architecture Overview

This section provides a high-level map of the codebase. Each subsystem is introduced briefly here and covered in depth in subsequent sections.

7.1 Module Summary

The Rust server is organized into the following source modules, all compiled as a single binary via `main.rs`.

File	Role	Details
`main.rs`	Entry point	CLI argument parsing, IP resolution, mode dispatch
`protocol.rs`	Wire protocol	Packet encoding/decoding, command builders, echo decompression
`connection.rs`	State machine	TCP auth, control channel, keepalives, response parsing (§14)
`web.rs`	Web server	Axum HTTP handlers, echo listener thread, recording, pattern engine (§9–§12)
`echo.rs`	Echo helpers	Rotation collection, CSV export, echo header parsing
`render.rs`	PPI renderer	Polar-to-Cartesian projection, color mapping, PNG encoding (§8)
`modes.rs`	CLI modes	Standalone operations: handshake, TX control, dump, render-live (§13)
`testing.rs`	Test harness	A/B command verification, 6-phase validation suite (§15)
`tests.rs`	Unit tests	`#[cfg(test)]` integration tests requiring live radar

Table 1: Source module summary.

7.2 System-Level Block Diagram

The following diagram shows the physical and network boundaries of the system. The radar hardware communicates with the Rust server over a dedicated Ethernet link, and the server exposes an HTTP interface to the browser.

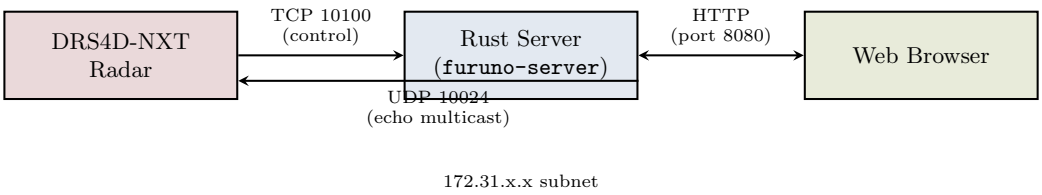


Figure 8: System-level block diagram showing radar hardware, Rust server, and browser.

7.3 Module-Level Diagram

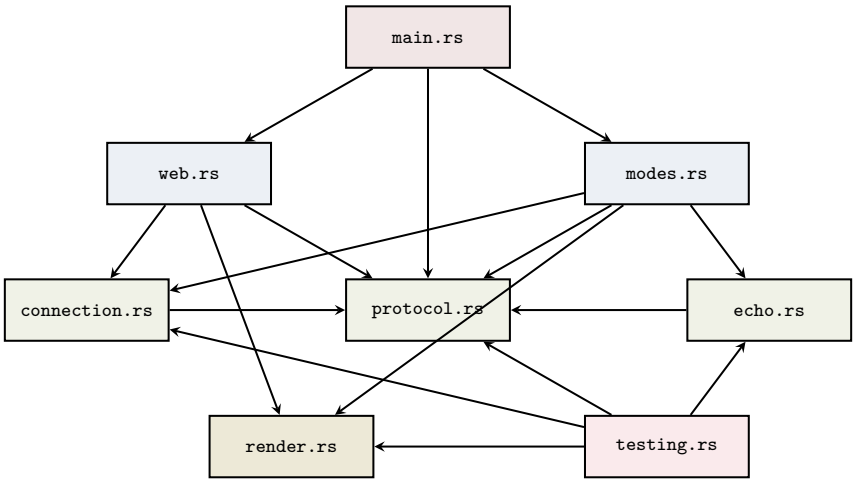


Figure 9: Module dependency diagram. Arrows point from dependent module to dependency.

7.4 Data Flow Diagram

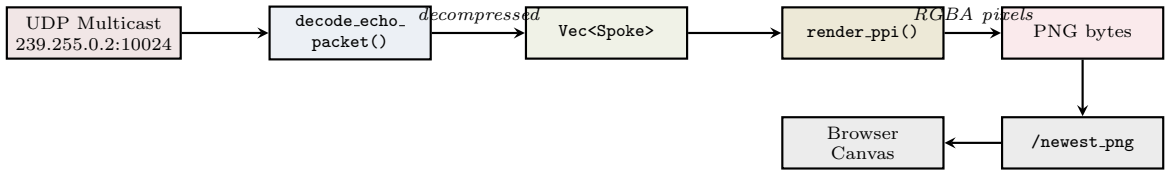


Figure 10: Echo data flow from UDP multicast to browser canvas.

7.5 Concurrency Model

The server uses a hybrid concurrency model combining Tokio async tasks with a dedicated OS thread.

The **Tokio async runtime** handles all HTTP request processing. Each Axum handler runs as an async task on the Tokio thread pool. Blocking radar operations (sending commands, waiting for acknowledgments) are wrapped in `tokio::task::spawn_blocking()` to avoid blocking the async executor.

A single **dedicated** `std::thread` runs the echo listener loop. This thread receives UDP multicast packets, decodes spokes, renders the PPI image, and optionally writes CSV/PNG files during recording. It runs independently of the HTTP handlers and communicates with them through shared state.

Synchronization primitives in the shared `AppState` struct coordinate access between the async handlers and the echo thread. `Mutex<RadarConnection>` provides exclusive access to the TCP control channel, ensuring only one thread sends commands at a time. `RwLock<Vec<u8>>` protects

the latest PNG image, allowing the echo thread to write and multiple HTTP handlers to read concurrently. Several `AtomicBool` flags (`tx_active`, `recording`, `pattern_stop`) provide lock-free coordination for frequently checked boolean state.

8 PPI Rendering

The PPI (Plan Position Indicator) renderer in `render.rs` converts a vector of decoded `Spoke` structs into a 640×640 pixel RGBA image. This is the most commonly modified subsystem when changing visual appearance, image resolution, or overlay elements.

8.1 Coordinate Transformation

Each spoke has an `angle_ticks` field representing a compass bearing (0 = north, increasing clockwise) in the range 0–8191 (8192 ticks per full rotation). The renderer converts this to a standard math angle (0 = east, increasing counter-clockwise) using the formula:

$$\theta_{\text{rad}} = \frac{\texttt{angle_ticks}}{8192} \times 2\pi \tag{1}$$

$$\theta_{\text{math}} = \frac{\pi}{2} - \theta_{\text{rad}} \tag{2}$$

For each spoke, the renderer walks outward from the image center along the radial line defined by θ_{math}, one pixel step at a time. At each step at radius r, the corresponding range bin index is calculated by linear interpolation:

$$\texttt{bin} = \left\lfloor \frac{r}{r_{\max}} \times N_{\text{bins}} \right\rfloor \tag{3}$$

where $r_{\max} = 318$ pixels (center minus 2-pixel margin) and $N_{\text{bins}} = 868$ (the standard number of range bins per spoke). The pixel coordinates are:

$$p_x = \texttt{center} + r \cdot \cos(\theta_{\text{math}}) \tag{4}$$

$$p_y = \texttt{center} - r \cdot \sin(\theta_{\text{math}}) \tag{5}$$

8.2 Color Mapping

Echo amplitudes arrive as 6-bit values (0–63) after the `wire >> 2` display conversion. The renderer scales these to 8-bit intensity and applies a warm-tinted color mapping:

$$\texttt{intensity} = \min\left(\frac{\texttt{amplitude} \times 255}{63},\ 255\right) \tag{6}$$

$$R = \texttt{intensity} \tag{7}$$

$$G = \left\lfloor \texttt{intensity} \times \frac{180}{255} \right\rfloor \approx 0.706 \times \texttt{intensity} \tag{8}$$

$$B = \left\lfloor \texttt{intensity} \times \frac{20}{255} \right\rfloor \approx 0.078 \times \texttt{intensity} \tag{9}$$

This produces a warm amber/orange color for strong returns, fading to dark red/brown for weak returns. Zero-amplitude bins are left as the black background (RGBA 0,0,0,255).

8.3 Overlay Elements

After all spokes are drawn, the renderer adds two overlay elements in dark gray (RGB 54, 54, 54).

Crosshairs are drawn as a horizontal and vertical line passing through the image center, spanning the full image width and height.

Range rings are drawn as four equally-spaced circles at radii $r_{\max}/4$, $r_{\max}/2$, $3r_{\max}/4$, and $r_{\max}$. Each ring is rasterized by stepping around its circumference with angular resolution proportional to its perimeter.

8.4 PNG Encoding

Two functions handle PNG output, both using the `png` crate.

`encode_png_bytes(pixels, width, height)` encodes the RGBA pixel buffer into an in-memory `Vec<u8>` containing the complete PNG file. This is used by the echo listener thread to update the `latest_png` shared state without touching the filesystem.

`write_png(path, pixels, width, height)` writes the PNG directly to a file via a buffered writer. This is used during recording (per-rotation PNG saves) and by CLI modes that produce output files.

Both functions encode as RGBA color type with 8-bit depth.

8.5 How to Modify

Image size. Change the `PPI_SIZE` constant (currently 640). All geometry is computed relative to this value, so no other changes are needed. Larger sizes produce smoother radial lines at the cost of increased memory and PNG encoding time.

Color palette. Modify the three lines in `render_ppi()` that assign to `pixels[idx]`, `pixels[idx+1]`, and `pixels[idx+2]`. For a green phosphor look, use $R = 0$, $G =$ `intensity`, $B = 0$. For a blue-white scheme, use $R =$ `intensity`$/2$, $G =$ `intensity`$/2$, $B =$ `intensity`.

New overlays. Add drawing code after the range ring loop in `render_ppi()`. The pixel buffer is a flat `Vec<u8>` in row-major RGBA order. The index for pixel (x, y) is $(y \times \texttt{size} + x) \times 4$, with offsets +0 for R, +1 for G, +2 for B, +3 for A.

9 Web Server

The web server in `web.rs` is the primary interface for operating the radar. It serves both the static HTML/CSS/JS files and a REST API for controlling radar parameters and retrieving data.

9.1 Framework

The server is built on **Axum 0.7**, a Rust web framework that runs on the **Tokio** async runtime. The `start_server()` function is annotated with `#[tokio::main]` to create the async runtime. All HTTP handlers are async functions, with blocking radar I/O operations wrapped in `tokio::task::spawn_blocking()` to prevent stalling the event loop.

Key dependencies: `axum` for routing and extraction, `tower-http` for static file serving, `serde` for query parameter deserialization, and `serde_json` for JSON responses.

9.2 Shared State

All server state lives in a single `AppState` struct, wrapped in `Arc` and passed to every handler via Axum's `State` extractor. The key fields and their synchronization primitives are listed below.

Field	Type	Purpose
`conn`	`Mutex<RadarConnection>`	Exclusive access to the TCP control channel
`latest_png`	`RwLock<Vec<u8>>`	Most recent PPI image (PNG-encoded bytes)
`tx_active`	`AtomicBool`	Whether the transmitter is currently on
`connected`	`AtomicBool`	Whether the radar connection is established
`recording`	`AtomicBool`	Whether CSV/PNG recording is active
`record_dir`	`RwLock<String>`	Current recording output directory path
`rotation_count`	`AtomicU64`	Total antenna rotations since server start
`pattern_stop`	`AtomicBool`	Signal to abort a running pattern
`pattern_running`	`AtomicBool`	Whether a pattern is currently executing
`pattern_status`	`RwLock<String>`	Human-readable pattern execution status
`settings`	`RwLock<CurrentSettings>`	Current radar configuration for CSV metadata
`setting_history`	`RwLock<Vec<SettingChange>>`	Change log for recording snapshots
`host_ip`	`Ipv4Addr`	Host IP (used by echo socket)
`start_time`	`Instant`	Server start time (for uptime calculation)

Table 2: `AppState` fields and synchronization.

9.3 Route Registration

Routes are registered in `start_server()` using Axum's `Router` builder. Each route maps a URL path to a handler function. All routes use the `GET` method except `/run_pattern_queue` which uses `POST` (for large JSON payloads). The router ends with a `fallback_service(ServeDir::new(web_dir))` that serves static files from the `web/` directory for any path that does not match a registered route.

Several legacy aliases exist for backward compatibility: `/begin_calibration` and `/run_ramp_pattern` both map to the same handler as `/run_pattern`.

9.4 Handler Pattern

All handlers follow a consistent pattern:

```
async fn handle_set_range(
    State(state): State<Arc<AppState>>,
    Query(params): Query<RangeParams>,
) -> String {
    // Build command from params
    // Send via spawn_blocking + send_cmd()
    // Update settings, return status string
}
```

The `State` extractor provides access to the shared `AppState`. The `Query` extractor deserializes URL query parameters into a typed struct. Handlers that modify radar settings also update the `CurrentSettings` in shared state and log a `SettingChange` entry for recording metadata.

The helper function `send_cmd()` wraps the blocking radar I/O pattern: it takes a command string and expected acknowledgment prefix, calls `send_and_read_ack()` inside `spawn_blocking()`, and returns a status message.

9.5 Echo Listener Thread

A dedicated `std::thread` (spawned at server startup) runs the `echo_listener()` function. This thread joins the UDP multicast group 239.255.0.2:10024 and continuously receives echo packets.

The listener maintains a running buffer of spokes for the current rotation. When it detects a rotation wrap (azimuth drops by more than 1000 ticks from the previous spoke), it renders the accumulated spokes into a PPI image via `render_ppi()`, encodes it to PNG via `encode_png_bytes()`, and stores the result in `latest_png`. If recording is active, it also writes a CSV and PNG file to the recording directory.

The listener sends keepalive packets every 5 seconds (via `try_lock()` on the connection mutex) and drains pending control channel data. When TX is not active, it sleeps in 100 ms intervals to avoid busy-waiting.

9.6 Static File Serving

The `tower_http::services::ServeDir` middleware serves the `web/` directory as a fallback for any path not matched by an explicit route. By default, the web directory is `../web` relative to the binary location. This can be overridden with the second positional argument to the `web` CLI mode.

The static files include `index.html`, `radar.html`, `settings.html`, `status.html`, `styles.css`, and `script.js`. The JavaScript in `script.js` handles page-specific logic including polling `/newest_png` every 500 ms, sending API requests, and managing pattern queue state.

9.7 How to Add a New Endpoint

To add a new endpoint, follow these steps.

1. **Define a parameter struct.** Create a `#[derive(Deserialize)]` struct with fields matching the expected query parameters. Use `#[serde(rename = "camelCase")]` for JavaScript-friendly names.
2. **Write the handler.** Create an async function taking `State(state): State<Arc<AppState>>` and `Query(params): Query<YourParams>`. Use `send_cmd()` for radar commands, or access shared state directly for read-only queries.
3. **Register the route.** Add a `.route("/your_endpoint", get(handle_your_endpoint))` line in the router chain in `start_server()`.

10 REST API Reference

All endpoints are verified against the source code in `web.rs`. Unless otherwise noted, all endpoints accept `GET` requests with URL query parameters and return plain text responses.

10.1 Power and Connection

Endpoint	Description
`/toggle_echo?mode=on|off`	Turn the transmitter on or off. Accepts optional `radarNo` parameter (ignored).
`/reconnect`	Drop the current connection and re-establish auth, control channel, and init. Resets TX state to off.

10.2 Settings

All settings endpoints accept `GET` requests with query parameters. During pattern execution, manual changes to pattern-controlled settings are rejected unless the pattern step uses `NaN` for that field.

Endpoint	Parameters
`/set_range`	`rangeIndex` (0–17)
`/set_pulse_width`	`pulse` (0–5: S1/S2/M1/M2/M3/L), optional `rangeIndex`
`/set_gain`	`status` (0=manual, 1=auto), `manualValue` (0–255), `autoValue`
`/set_acsea`	`status`, `manual` (0–255), `auto`, `mode` (0=Advanced, 1=Coastal)
`/set_acrain`	`status`, `manual` (0–255), `auto`, `ftcMode` (0/1/2), `rainMode` (0/1/2)
`/set_bearing_resolution`	`mode` (0–3: Off/Low/Med/High)
`/set_antenna_revolution`	`rollSpeed` (0=Lo, 1=Hi, 2=Auto), `hiRpm` (0/1)
`/set_near_stc`	`value` (0–4)
`/set_middle_stc`	`value` (0–3)
`/set_far_stc`	`value` (0–2)
`/set_main_bang`	`value` (0–255)
`/set_interference_rejection`	`enabled` (0=off, 1=on)
`/set_bird_mode`	`level` (0–3: Off/Low/Med/High)
`/set_doppler`	`mode` (0=Off, 1=Internal heading, 2=External heading)

10.3 State Queries

Endpoint	Description
/get_current	Query a specific setting from the radar. command parameter selects the setting: TX, RANGE, PULSE, GAIN, CLUTTER, ANTENNA, STC, MAINBANG, IR, BIRDMODE, DOPPLER, SWEEP. Optional format=json for JSON response (STC only).
/get_system_info	Returns JSON with full system state: connection status, TX state, firmware, IPs, recording state, rotation count, uptime, pattern status, current settings, memory and storage usage.
/system_metrics	Returns JSON with host system metrics: per-core CPU usage, load average, memory usage, storage usage, network throughput, and CPU temperature.

10.4 Echo Image

GET /newest_png returns the most recent PPI image as image/png with Cache-Control: no-cache. If no echo data has been received yet, returns a 1×1 black pixel PNG as a placeholder. The browser JavaScript polls this endpoint every 500 ms to update the radar display.

10.5 Recording

GET /toggle_recording?mode=start|stop starts or stops recording. When started, creates a timestamped directory under recordings/ with csv/ and png/ subdirectories, and writes a settings.json snapshot. Each completed rotation during recording produces a CSV and PNG file in the respective subdirectories.

10.6 Pattern Execution

Endpoint	Method	Description
/run_pattern	GET	Execute a single pattern. steps (semicolon-separated), repeat count, optional label.
/run_pattern_queue	POST	Execute a queue of patterns. JSON body with base64-encoded payload containing pattern items.
/stop_pattern	GET	Set the pattern_stop flag to abort the current pattern.
/pattern_status	GET	Returns a JSON object with current pattern execution status, active step info, and queue progress.

Legacy aliases `/begin_calibration` and `/run_ramp_pattern` both map to the same handler as `/run_pattern`.

10.7 Stub Endpoints

`/set_sweep_total` and `/set_interpolation` are registered routes that return `"OK"` without performing any action. These exist for compatibility with the web interface JavaScript, which sends requests to these endpoints. The sweep total is fixed at the PPI resolution (640×640) and interpolation is handled client-side.

11 Recording System Internals

This section describes how the recording system works internally. For user-facing recording instructions, see §5.

11.1 Echo Listener Integration

Recording is triggered by the echo listener thread, not by the HTTP handlers. When the `recording` flag is set to `true` and the echo listener detects a rotation wrap (azimuth drops by more than 1000 ticks), it writes the completed rotation's data to disk before clearing the spoke buffer.

The echo listener checks `state.recording.load(Ordering::Relaxed)` on every rotation boundary. If recording is active and `record_dir` is non-empty, it writes both a CSV file and a PNG file for that rotation. This tight coupling with the echo thread ensures every rotation is captured without gaps.

11.2 Directory Creation

When recording starts, a timestamped directory is created under `recordings/`. The directory name encodes the date and time (e.g., `recordings/20260307_143022/`). Two subdirectories are created inside: `csv/` for spoke data files and `png/` for rendered PPI images. The path is stored in `state.record_dir` so the echo listener thread can find it.

If the recording was triggered by a pattern queue, the directory is created per-pattern-item, with each item getting its own timestamped directory and `settings.json` snapshot.

11.3 CSV Writing

The `write_echo_csv()` function in `web.rs` writes a single rotation's data to a CSV file. Each file begins with `#`-prefixed metadata lines containing the current radar settings at the time of capture. These metadata lines record every field from `CurrentSettings`: range index and label, gain mode and value, pulse width, sea clutter, rain clutter, RezBoost level, interference rejection state, bird mode, Doppler mode, antenna speed, main bang suppression, near/middle/far STC values.

After the metadata header, a column header line defines the fields: `spoke_index`, `angle_ticks`, `angle_deg`, `heading_ticks`, `num_bins`, followed by `bin_0` through `bin_867`. Each subsequent line contains one spoke's data. Spokes with fewer than the maximum number of bins are zero-padded.

11.4 Settings Snapshots

A `settings.json` file is written to the recording directory when recording starts. The schema depends on how recording was initiated.

Manual recording produces a `ManualRecordingSettingsSnapshot` containing the mode (`"manual"`), the recording directory path, the full `CurrentSettings` object, and the setting change history accumulated since the server started.

Pattern recording produces a `PatternRecordingSettingsSnapshot` containing the mode (`"pattern"`), the recording directory, `CurrentSettings`, a `PatternInfo` object describing the active pattern (label, repeat count, step definitions, queue position), and the change history.

11.5 Setting Change History

Every time a radar setting is changed through the web API, a `SettingChange` entry is appended to the `setting_history` vector. Each entry records a timestamp string, the setting name (e.g., `"range_index"`, `"gain_value"`), and the new value as a JSON value. This history accumulates for the entire server session and is included in every `settings.json` snapshot, providing a complete audit trail of all parameter changes.

12 Pattern Execution Engine

This section covers the internal mechanics of pattern execution. For user-facing pattern documentation, see §6.

12.1 Step Parsing

The `parse_pattern_steps()` function in `web.rs` converts a semicolon-separated string of pattern steps into a vector of `ControlStep` structs. Each step is split by commas, and the number of fields determines the format.

Seven-field steps are parsed as the simple format: all fields are required integers for range, gain, pulse, RezBoost, speed, interpolation, and repeat count. Thirteen-field steps use the advanced format: the first six fields plus sea clutter, rain clutter, main bang, interference rejection, bird mode, doppler, and repeat. In the advanced format, any field except repeat can be `NaN` to leave that parameter unchanged.

The parser validates field counts (must be exactly 7 or 13), checks numeric ranges, and returns descriptive error messages identifying the offending step and field.

12.2 Differential Command Generation

When a pattern step is applied, the engine compares each field of the new step against the current settings. Only parameters that differ from the current state are sent to the radar. This minimizes unnecessary command traffic and avoids redundant acknowledgment waits. The comparison uses the `CurrentSettings` struct, which tracks what the server believes the radar's current configuration to be.

For `NaN` fields in the advanced format, no comparison or command is generated at all — the parameter is simply skipped.

12.3 Single Pattern Execution

The `execute_pattern()` function runs a single pattern. For each step, it applies the settings via differential commands, then waits for the specified number of full antenna rotations by polling `state.rotation_count`. Between rotation checks, it sleeps briefly to avoid busy-waiting.

The function updates `pattern_status` at each step transition with a human-readable message (e.g., `"Step 3/7: waiting for rotation 2/5"`). If the pattern has a positive repeat count, the entire sequence of steps is repeated that many times. A repeat count of -1 loops indefinitely until stopped.

12.4 Queue Execution

The `handle_run_pattern_queue()` handler receives a POST request with a JSON body containing a base64-encoded payload. The payload is decoded into a list of pattern items, each with its own step data, repeat count, and label.

The handler iterates through queue items sequentially. For each item, it starts a new recording session (creating a fresh timestamped directory), executes the pattern, and stops recording when the pattern completes. The active queue position is tracked in `pattern_status` so the web interface can display progress (e.g., `"Queue item 2/5"`).

12.5 NaN Override Logic

The `pattern_step_allows_override()` function determines whether a manual setting change should be allowed during pattern execution. When a pattern is running, manual changes through the web API are normally rejected to prevent interference. However, if the currently active pattern step has `NaN` for a particular field, that field remains "unlocked" and can be changed manually.

This allows operators to adjust parameters like gain or clutter during a pattern run without disrupting the automated sequence. The function checks the active step's `ControlStep` fields: if the field corresponding to the requested setting is `None` (parsed from `NaN`), the override is allowed.

12.6 Stop Mechanism

The `pattern_stop AtomicBool` flag is the sole mechanism for aborting a running pattern. It is checked at two points in the execution loop: between steps (before applying the next step's settings) and between rotation waits (before sleeping for the next rotation).

The `/stop_pattern` endpoint sets this flag to `true`. The pattern execution loop detects the flag, updates `pattern_status` to indicate cancellation, resets the flag, and exits. The stop takes effect within one rotation period (typically 1.25–2.5 seconds depending on antenna speed).

13 CLI Modes (All Other Modes)

This section documents the CLI modes used for development, debugging, and testing. The `web` and `discover` modes are documented in Part I (§?? and §??).

13.1 Handshake

`furuno-server handshake` is the default mode when no mode argument is provided. It performs the full connection sequence — authentication, control channel establishment, and initialization — then prints the firmware version string and exits. This is the primary tool for verifying that the radar is reachable and the protocol implementation is working.

13.2 TX On / TX Off

`furuno-server tx-on` and `furuno-server tx-off` perform the full connection sequence and then send the transmitter on or off command. The TX ON command (`$S69,2,...`) is always accompanied by STC reset commands (`$S85,0`, `$S86,0`, `$S87,0`). After sending the command, echo data should start (or stop) flowing on the multicast group.

13.3 Full Cycle

`furuno-server full` connects, enables TX, collects two full antenna rotations of echo data (using the second rotation to avoid partial data from the first), disables TX, and saves the captured data as both a CSV file and a rendered PPI PNG. Output files are named with timestamps (e.g., `radar_20260307_143022.csv`).

13.4 Render Live

`furuno-server render-live [output.png]` captures one live rotation and renders it to a PNG file. The optional argument specifies the output path (default: `radar_live.png`). A companion CSV file is also saved alongside the PNG. This mode is useful for quickly capturing a single radar image without starting the web server.

13.5 Echo Listen

`furuno-server echo` passively listens for multicast echo packets on 239.255.0.2:10024 without performing authentication or sending any commands. It prints packet headers (sequence number, azimuth, timestamp, flags, config) for the first 10 packets and every 100th packet thereafter. This mode does not require exclusive radar access and can run while another client is controlling the radar.

13.6 Command Test

`furuno-server command` connects to the radar and sends a predefined sequence of test commands (range change, gain change, STC adjustment, RezBoost toggle), printing the acknowledgment responses. The commands are sent in pairs: set a value, then restore the default. This mode is useful for verifying that individual commands produce the expected responses.

13.7 Dump Raw

`furuno-server dump-raw [count]` captures the specified number of raw echo packets (default: 5) and prints detailed hex dumps. For each packet, it shows the 18-byte header, identifies spoke section boundaries within the packet body, and attempts to decode each section's RLE structure, reporting the total bin count and RLE control bytes. This is the primary tool for debugging echo encoding issues.

13.8 Test Commands

`furuno-server test-commands` runs the A/B command verification suite in `testing.rs`. For each of 18 radar parameters, it captures a "before" PPI image, sends a command to change the parameter, captures an "after" PPI image, then restores the original value. Results are saved to a `test_output/` directory with before/after PNG pairs and a summary table showing whether each command was acknowledged and what echo statistics changed.

13.9 Validate

`furuno-server validate` runs the comprehensive 6-phase validation suite. This is the most thorough automated test of the system. Output goes to a timestamped directory.

1. **Connection & Init** — full handshake, firmware query.
2. **TX ON** — enable transmitter, verify echo data flows.
3. **Settings A/B/A** — for 14 parameters, set value A, read back, set value B, read back, set value A again. Checks that readbacks match and that A≠B (anti-stale check).
4. **Echo Impact (Gain)** — compare mean echo amplitude at low gain (10) vs high gain (90).
5. **Echo Impact (Range)** — compare sweep parameters at 1/8 NM vs 8 NM.
6. **Restore & TX OFF** — reset all settings to defaults and disable transmitter.

A text report file summarizes PASS/FAIL/SKIP counts. CSV and PNG files are saved for the echo impact tests.

14 Connection State Machine

The `RadarConnection` struct in `connection.rs` implements a five-state machine that enforces correct sequencing of the connection protocol.

14.1 States and Transitions



Figure 11: Connection state machine. Each transition corresponds to a method call on `RadarConnection`.

The five states are:

1. **Disconnected** — initial state. No TCP connections exist.
2. **Authenticated** — the 44-byte auth payload has been sent to TCP port 10010 and the radar responded with status byte 0x64. The auth TCP connection is then closed.
3. **ControlConnected** — a TCP connection to port 10100 has been established with `TCP_NODELAY` enabled, and the `$N00,CLIENTCONNECTION` message has been received.
4. **Initialized** — the firmware query (`$R96`) and all ∼184 init register reads have been sent and their responses received.
5. **Transmitting** — the TX ON command (`$S69,2,...`) has been sent. Echo data is flowing on multicast.

Transitions are enforced by state checks: `connect_control()` requires at least `Authenticated`, `init()` requires `ControlConnected`, and `tx_on()`/`tx_off()` require `Initialized` or `Transmitting`. Calling a method in the wrong state returns `RadarError::InvalidState`.

14.2 RadarError Enum

Variant	Meaning
`AuthFailed(String)`	Auth response was absent or had unexpected status byte
`ConnectionFailed(String)`	TCP connection to radar could not be established
`IoError(std::io::Error)`	Low-level I/O error (read/write failure)
`Timeout`	Operation exceeded its deadline
`InvalidState{expected, actual}`	Method called in wrong connection state
`ProtocolError(String)`	Malformed response or unexpected protocol behavior

Table 7: `RadarError` variants.

14.3 Timeout Values

Operation	Timeout	Notes
Authentication TCP connect	5 s	`TcpStream::connect_timeout`
Auth response read	3 s	Socket read timeout
Control channel TCP connect	5 s	`TcpStream::connect_timeout`
Initial message read	2 s	Waiting for `$N00,CLIENTCONNECTION`
Init register responses	3 s	Bulk read after sending ∼184 queries
Command acknowledgment	3 s	`send_and_read_ack` default
Subsequent read (within burst)	200 ms	Short timeout for trailing messages
Keepalive interval	5 s	Echo thread sends if elapsed
Echo socket read	500 ms	UDP receive timeout

Table 8: Timeout values used throughout the connection lifecycle.

14.4 Graceful Degradation

The web server starts even if the initial radar connection fails. In `start_server()`, if `connect_full()` returns an error, the `connected` flag is set to `false` and a warning is printed. The HTTP server still binds its port and serves the web interface. All API endpoints that require a radar connection check the `connected` flag and return an error message directing the user to the `/reconnect` endpoint.

The `/reconnect` endpoint creates a fresh `RadarConnection`, drops the old one, and attempts `connect_full()` again. On success, it updates the `connected` flag and resets `tx_active` to false.

14.5 Keepalive Handling

The radar expects periodic keepalive packets on the control channel. If keepalives stop, the radar drops the client connection after approximately 30 seconds.

In web mode, the echo listener thread handles keepalives. Every 5 seconds (checked via `last_keepalive.elapsed()`) it acquires the connection mutex via `try_lock()` (non-blocking to avoid deadlocking with HTTP handlers), sends the 8-byte binary keepalive packet, and calls `drain_control()` to discard any pending control channel data.

In CLI modes, keepalives are sent explicitly by calling `conn.send_keepalive()` inside the echo collection loop. The `drain_control()` call prevents the TCP receive buffer from filling up with unsolicited telemetry messages.

15 Testing & Validation

All tests require a live radar connected to the host. There is no mock or simulation mode.

15.1 Running Tests

```
cd furuno-server
cargo test -- --test-threads=1
```

The `--test-threads=1` flag is required because all tests share a single radar connection, and the radar supports only one authenticated client at a time. Running tests in parallel causes connection conflicts.

15.2 Test Categories

The test suite in `tests.rs` contains 26 integration tests organized into five categories.

15.2.1 Connection Tests (6)

These verify the connection state machine: authentication succeeds with status 0x64, the control channel receives `$N00,CLIENTCONNECTION`, initialization completes with firmware data, keepalives are sent and acknowledged, and the echo multicast socket can be created and joined.

15.2.2 TX Tests (3)

TX on/off cycling verifies that `tx_on()` transitions to the `Transmitting` state and `tx_off()` returns to `Initialized`. Echo reception verifies that multicast packets arrive after TX ON. Full rotation collection verifies that the rotation wrap detection correctly identifies a complete 360 sweep.

15.2.3 Settings Tests (14)

Each settings test follows a set-and-read pattern: send a `$S` command to change a parameter, then send the corresponding `$R` query and verify the response contains the expected value. Parameters tested include range, gain, pulse width, sea clutter, rain clutter, RezBoost, antenna revolution, near/middle/far STC, main bang, interference rejection, bird mode, and Doppler.

15.2.4 Echo Impact Tests (2)

These tests verify that parameter changes produce measurable differences in echo data. The gain impact test compares mean echo amplitude at low vs high gain. The range impact test compares sweep parameters (sweep length, spoke count) at short vs long range. These are environment-dependent and may produce SKIP/WARN results if the radar is not receiving returns.

15.2.5 Render Integration Test (1)

The render integration test exercises the complete pipeline: receive UDP echo packets, decode them into `Spoke` structs, render via `render_ppi()`, encode to PNG via `encode_png_bytes()`, and verify the result is a valid PNG with non-zero size.

15.3 A/B Command Testing

The `test-commands` CLI mode (`testing.rs`) provides a more visual form of testing. For each of 18 parameters, it captures a full rotation PPI image before and after applying a command, then saves both as PNG files. An operator can visually compare the before/after pairs to verify command effects.

Each test follows a setup–before–apply–after–restore sequence. The setup command establishes a known baseline. The "before" capture collects one rotation at the baseline. The "apply" command changes the parameter. The "after" capture collects one rotation with the new setting. The restore command returns to the baseline.

Results are saved to `test_output/` with filenames like `01_range_2nm_BEFORE.png` and `01_range_2nm_AFTER.png`. A summary table at the end shows ACK status and echo statistics for each test.

15.4 Validation Suite

The `validate` CLI mode runs the comprehensive 6-phase test described in §13.9. It produces a timestamped output directory containing CSV files, PNG captures, and a `report.txt` with PASS/FAIL/SKIP counts and detailed per-test output. The validation suite tests 14 parameters in A/B/A fashion (3 checks each = 42 assertions), plus connection, TX, and echo impact tests, for a total of approximately 47 individual assertions.

Part III

Echo Data Pipeline

16 Echo Data Format

16.1 Packet Structure

16.2 Header Field Breakdown

16.3 Echo Metadata Extraction

16.4 Spoke Sub-Header

16.5 Multi-Spoke Packing

16.6 Azimuth and Heading

16.7 Typical Packet Characteristics

17 Echo Encoding Types

17.1 Encoding 0: Raw Uncompressed

17.2 Encoding 1: RLE with Repeat Marker

17.3 Encoding 2: Delta RLE with Previous Spoke Reference

17.4 Encoding 3: 2-Bit RLE with Dual Reference

17.5 Encoding Selection

17.6 Display Value Conversion

17.7 Rotation Detection

18 Doppler Processing

18.1 Status Field Encoding

18.2 Echo Byte Encoding

18.3 Downstream Usage

18.4 Practical Notes

19 Dual-Pulse Interleaving

19.1 Pulse Types

19.2 Range Dependence

19.3 Rez Boost Effect

19.4 Spoke Handling

Part IV

Network & Wire Protocol

20 Network Architecture

20.1 Topology

20.2 Port Map

20.3 Auto-Discovery and Network Configuration

20.3.1 Platform-Specific IP Alias

21 Control Protocol

21.1 Message Format

21.2 Binary Keepalive

21.3 Binary vs ASCII Disambiguation

21.4 Response Types

21.5 Response Parsing Pipeline

21.6 Command Acknowledgment

21.7 Control Channel Drain

21.8 Initialization Sequence

21.8.1 Phase 1: Discovery

21.8.2 Phase 2: Authentication

21.8.3 Phase 3: Control Channel

21.8.4 Phase 4: Firmware Query

21.8.5 Phase 5: Register Dump

21.8.6 Phase 6: Steady State

22 Command Reference

22.1 TX State ($S69)

22.2 Range ($S62)

22.3 Gain ($S63)

22.4 Sea Clutter ($S64)

22.5 Rain Clutter ($S65)

22.6 Pulse Width ($S68)

22.7 STC Near ($S85)

22.8 STC Middle ($S86)

22.9 STC Far ($S87)

22.10 Main Bang Suppression ($S83)

22.11 Antenna Height ($S84)

22.12 Antenna Revolution ($S89)

22.13 Interference Rejection ($S67, param[2]=2)

22.14 Noise Reduction ($S67, param[2]=3)

22.15 Rez Boost ($SEE)

22.16 Bird Mode ($SED)

22.17 Doppler / Target Analyzer ($SEF)

22.18 No-Transmit Sectors ($S77)

22.19 Query Commands

23 Periodic Telemetry

23.1 Telemetry Message Table

23.2 Telemetry Processing

23.3 Parsed vs Discarded Data

24 Proprietary Library Reference

24.1 Module Breakdown

24.2 Shared Command IDs